EJERCICIOS COMPLETOS DE PRUEBAS ESTRUCTURALES

Ejercicio 1 — Validador de matrículas y cálculo de tasa

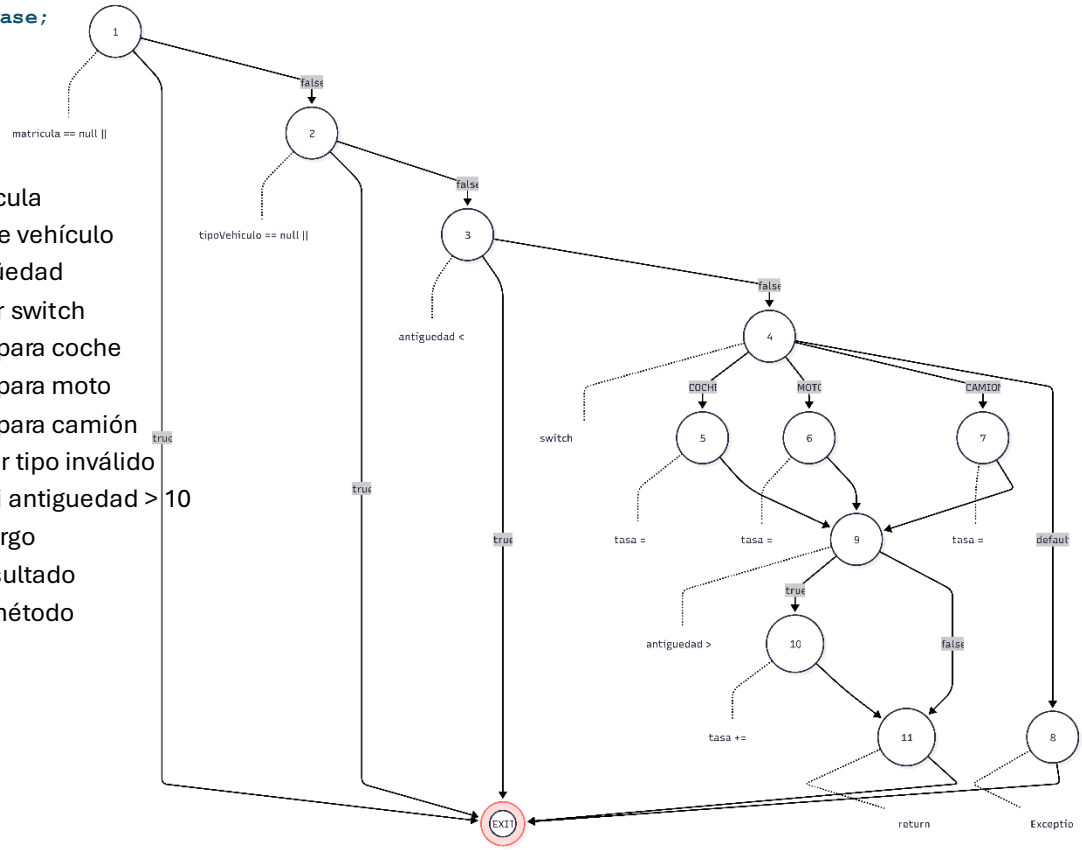
Valida una matrícula, calcula una tasa según tipo de vehículo y aplica un recargo por antigüedad.

1. Código Java

```java
public class RegistroVehiculos {
    public static double calcularTasa(String matricula, String tipoVehiculo, int antiguedad) {
        if (matricula == null || matricula.isBlank()) {
            throw new IllegalArgumentException("La matrícula no puede ser nula o vacía");
        }
        if (tipoVehiculo == null || tipoVehiculo.isBlank()) {
            throw new IllegalArgumentException("El tipo de vehículo no puede ser nulo o vacío");
        }
        if (antiguedad < 0) {
            throw new IllegalArgumentException("La antigüedad no puede ser negativa");
        }
        double tasaBase;
        switch (tipoVehiculo.toUpperCase()) {
            case "COCHE":
                tasaBase = 100;
                break;
            case "MOTO":
                tasaBase = 60;
                break;
            case "CAMION":
                tasaBase = 150;
                break;
            default:
                throw new IllegalArgumentException("Tipo de vehículo no válido");
        }
        if (antiguedad > 10) {
            tasaBase += 20;
        }
        return tasaBase;
    }
}
```

2. Grafo de flujo

- **N1**: validar matrícula
- **N2**: validar tipo de vehículo
- **N3**: validar antigüedad
- **N4**: selección por switch
- **N5**: asignar tasa para coche
- **N6**: asignar tasa para moto
- **N7**: asignar tasa para camión
- **N8**: excepción por tipo inválido
- **N9**: comprobar si antiguedad > 10
- **N10**: aplicar recargo
- **N11**: devolver resultado
- **EXIT**: salida del método



3. Complejidad ciclomática

Método 1: por predicados o decisiones

Las decisiones reales del grafo son:

- **N1**: matricula == null || blank
- **N2**: tipoVehiculo == null || blank
- **N3**: antiguedad < 0
- **N4**: switch(tipoVehiculo) -> se puede tomar como 1 (todo el switch) o como 4.
- **N9**: antiguedad > 10

Si tratamos el switch como una estructura de decisión, la complejidad queda: **V(G) = 5**

Si tratamos la selección del switch y cada opción tenremos **V(G) = 8.**

Método 2: por fórmula de McCabe

La fórmula es: $V(G) = E - N + 2P$

donde:

- **E** = número de aristas
- **N** = número de nodos
- **P** = número de componentes conexos del grafo

Aquí hay un único componente, por tanto: $P = 1$

Nodos

Contamos:

- N1, N2, N3, N4, N5, N6, N7, N8, N9, N10, N11, EXIT

Total: $N = 12$

Aristas

N1 → EXIT, N1 → N2, N2 → EXIT, N2 → N3, N3 → EXIT, N3 → N4, N4 → N5, N4 → N6, N4 → N7, N4 → N8, N5 → N9, N6 → N9, N7 → N9, N8 → EXIT, N9 → N10, N9 → N11, N10 → N11, N11 → EXIT

Total: $E = 18$

Aplicando la fórmula: $V(G) = 18 - 12 + 2 = 8$

Aclaración importante

Aquí aparece una cuestión muy relevante en docencia: **el resultado depende del nivel de granularidad del grafo**.

Tu grafo:

- separa cada rama del switch como nodos distintos
- separa el nodo de excepción N8
- incluye EXIT explícito

Por eso, **el valor formal del grafo que has dibujado es 8**.

4. Caminos básicos

Al tener complejidad ciclomática **8**, debemos identificar **8 caminos independientes**.

Un conjunto válido de caminos básicos es el siguiente:

- **P1** N1 → EXIT Caso en que la matrícula es inválida.
- **P2** N1 → N2 → EXIT Matrícula válida, pero tipo de vehículo inválido o vacío.
- **P3** N1 → N2 → N3 → EXIT Matrícula y tipo correctos, pero antigüedad negativa.
- **P4** N1 → N2 → N3 → N4 → N8 → EXIT Todas las validaciones iniciales correctas, pero el switch entra en default.
- **P5** N1 → N2 → N3 → N4 → N5 → N9 → N11 → EXIT Tipo COCHE, sin recargo por antigüedad.
- **P6** N1 → N2 → N3 → N4 → N5 → N9 → N10 → N11 → EXIT Tipo COCHE, con recargo por antigüedad.
- **P7** N1 → N2 → N3 → N4 → N6 → N9 → N11 → EXIT Tipo MOTO, sin recargo.
- **P8** N1 → N2 → N3 → N4 → N7 → N9 → N11 → EXIT Tipo CAMION, sin recargo.

5. Casos de prueba

Caso	matrícula	tipoVehiculo	antigüedad	Resultado esperado	Camino
CP1	null	COCHE	5	Excepción	P1
CP2	"1111AAA"	""	5	Excepción	P2
CP3	"2222BBB"	MOTO	-1	Excepción	P3
CP4	"3333CCC"	BUS	5	Excepción	P4
CP5	"4444DDD"	COCHE	5	100.0	P5
CP6	"5555EEE"	COCHE	12	120.0	P6
CP7	"6666FFF"	MOTO	5	60.0	P7
CP8	"7777GGG"	CAMION	5	150.0	P8

6. JUnit 5 clásico

```java
import org.junit.jupiter.api.AfterAll;
import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeAll;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class RegistroVehiculosTest {
    private static int contadorTests;
    private String matriculaValida;

    @BeforeAll
    static void inicializarSuite() {
        contadorTests = 0;
        System.out.println("=== Inicio de la batería de tests de RegistroVehiculos ===");
    }

    @AfterAll
    static void finalizarSuite() {
        System.out.println("=== Fin de la batería de tests ===");
        System.out.println("Tests ejecutados: " + contadorTests);
    }

    @BeforeEach
    void prepararEntorno() {
        matriculaValida = "1234ABC";
        contadorTests++;
    }

    @AfterEach
    void limpiarEntorno() {
        matriculaValida = null;
    }

    @Test
    void testCP1MatriculaNulaLanzaExcepcion() {
        IllegalArgumentException ex = assertThrows(
                IllegalArgumentException.class,
                () -> RegistroVehiculos.calcularTasa(null, "COCHE", 5)
        );
        assertNotNull(ex);
        assertTrue(ex.getMessage().contains("matrícula"));
    }

    @Test
    void testCP2TipoVacioLanzaExcepcion() {
        IllegalArgumentException ex = assertThrows(
                IllegalArgumentException.class,
                () -> RegistroVehiculos.calcularTasa(matriculaValida, "", 5)
        );
        assertFalse(ex.getMessage().isEmpty());
    }

    @Test
    void testCP3AntiguedadNegativaLanzaExcepcion() {
        IllegalArgumentException ex = assertThrows(
                IllegalArgumentException.class,
                () -> RegistroVehiculos.calcularTasa(matriculaValida, "MOTO", -1)
        );
        assertEquals("La antigüedad no puede ser negativa", ex.getMessage());
    }

    @Test
    void testCP4TipoNoValidoEnSwitchLanzaExcepcion() {
        IllegalArgumentException ex = assertThrows(
                IllegalArgumentException.class,
                () -> RegistroVehiculos.calcularTasa(matriculaValida, "BUS", 5)
        );
        assertTrue(ex.getMessage().contains("no válido"));
    }
```

```
 @Test
    void testCP5CocheSinRecargo() {
        double resultado = RegistroVehiculos.calcularTasa(matriculaValida, "COCHE", 5);
        assertEquals(100.0, resultado);
        assertTrue(resultado >= 100.0);
    }

    @Test
    void testCP6CocheConRecargo() {
        double resultado = RegistroVehiculos.calcularTasa(matriculaValida, "COCHE", 12);
        assertEquals(120.0, resultado);
        assertNotEquals(100.0, resultado);
    }

    @Test
    void testCP7MotoSinRecargo() {
        double resultado = RegistroVehiculos.calcularTasa(matriculaValida, "MOTO", 5);
        assertEquals(60.0, resultado);
        assertTrue(resultado < 100.0);
    }

    @Test
    void testCP8CamionSinRecargo() {
        double resultado = RegistroVehiculos.calcularTasa(matriculaValida, "CAMION", 5);
        assertEquals(150.0, resultado);
        assertTrue(resultado > 100.0);
    }
}
```

7. Cobertura esperada en IntelliJ IDEA

Si ejecutas la clase de test con:

Run 'RegistroVehiculosTest' with Coverage

deberías obtener una cobertura muy alta, y en la práctica completa para este método.

Qué se cubre

Validaciones iniciales

- matrícula inválida
- tipo inválido o vacío
- antigüedad negativa

switch

- rama COCHE
- rama MOTO
- rama CAMION
- rama default

Condición final

- antiguedad > 10 verdadera
- antiguedad > 10 falsa

Resultado esperable

- **Cobertura de líneas**: 100%
- **Cobertura de métodos**: 100%
- **Cobertura de ramas**: 100% en el método calcularTasa

En el editor de IntelliJ deberías ver prácticamente todo el método en verde, y en la ventana de cobertura aparecerán cubiertas todas las ramas del if y del switch.

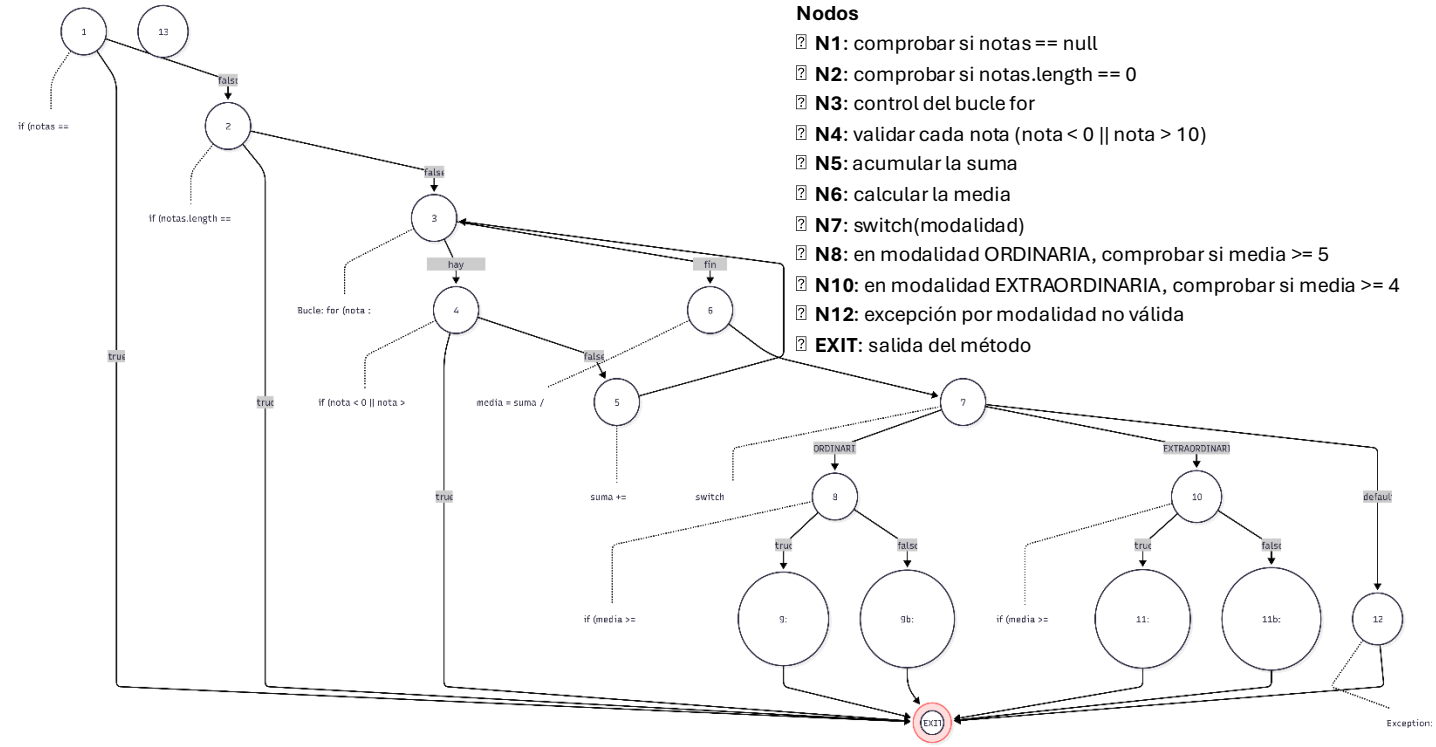
Ejercicio 2 — Analizador de notas con bucle y switch

Aquí ya aparece un **bucle** y un switch, además de varias validaciones. Es un módulo muy típico de programación.

1. Código Java

```
public class EvaluadorAsignatura {

    public static String evaluar(double[] notas, String modalidad) {
        if (notas == null) {
            throw new IllegalArgumentException("El array de notas no puede ser nulo");
        }

        if (notas.length == 0) {
            throw new IllegalArgumentException("Debe haber al menos una nota");
        }

        double suma = 0;

        for (double nota : notas) {
            if (nota < 0 || nota > 10) {
                throw new IllegalArgumentException("Nota fuera de rango");
            }
            suma += nota;
        }

        double media = suma / notas.length;

        switch (modalidad.toUpperCase()) {
            case "ORDINARIA":
                if (media >= 5) {
                    return "APROBADO";
                } else {
                    return "SUSPENSO";
                }
            case "EXTRAORDINARIA":
                if (media >= 4) {
                    return "APROBADO";
                } else {
                    return "SUSPENSO";
                }
            default:
                throw new IllegalArgumentException("Modalidad no válida");
        }
    }
}
```

2. Grafo de flujo



Nodos

- **N1**: comprobar si notas == null
- **N2**: comprobar si notas.length == 0
- **N3**: control del bucle for
- **N4**: validar cada nota (nota < 0 || nota > 10)
- **N5**: acumular la suma
- **N6**: calcular la media
- **N7**: switch(modalidad)
- **N8**: en modalidad ORDINARIA, comprobar si media >= 5
- **N10**: en modalidad EXTRAORDINARIA, comprobar si media >= 4
- **N12**: excepción por modalidad no válida
- **EXIT**: salida del método

3. Complejidad ciclomática

Método 1: por decisiones

Las decisiones estructurales son:

- **N1**: if (notas == null)
- **N2**: if (notas.length == 0)
- **N3**: control del bucle for
- **N4**: if (nota < 0 || nota > 10)
- **N7**: switch(modalidad), 1 si solo contamos el switch, **3 si sumamos las dos opciones**.
- **N8**: if (media >= 5)
- **N10**: if (media >= 4)

Por tanto: $V(G) = 7$ o V(G) = 9.

Método 2: por fórmula de McCabe

La fórmula es: $V(G) = E - N + 2P$

donde:

- **E** = aristas
- **N** = nodos
- **P** = componentes conexos

Aquí: $P = 1$

Conteo de nodos

En el grafo aparecen: Total: $N = 15$

Conteo de aristas

N1 → EXIT, N1 → N2, N2 → EXIT, N2 → N3, N3 → N4, N4 → EXIT, N4 → N5, N5 → N3, N3 → N6, N6 → N7, N7 → N8, N8 → N9, N8 → N9bis, N7 → N10, N10 → N11, N10 → N11bis, N7 → N12, N9 → EXIT, N9bis → EXIT, N11 → EXIT, N11bis → EXIT, N12 → EXIT

Total: $E = 22$

Aplicando la fórmula: $V(G) = 22 - 15 + 2 = 9$

Aclaración importante

Igual que en el ejercicio 1, aquí hay dos niveles de análisis:

Complejidad didáctica simplificada del método $V(G) = 7$

Complejidad del grafo detallado que has dibujado $V(G) = 9$

¿Por qué sube a 9 en el grafo detallado?

Porque has separado explícitamente:

- los distintos finales del switch
- los nodos de salida por aprobado/suspenso
- el default
- el comportamiento cíclico del bucle

Eso implica que debemos trabajar con **9 caminos independientes**.

4. Caminos básicos

Un conjunto válido de **9 caminos básicos** es el siguiente.

- **P1** N1 → EXIT Caso en que el array de notas es null.
- **P2** N1 → N2 → EXIT El array existe pero está vacío.
- **P3** N1 → N2 → N3 → N4 → EXIT El array no es nulo ni vacío, entra en el bucle y aparece una nota fuera de rango.
- **P4** N1 → N2 → N3 → N4 → N5 → N3 → N6 → N7 → N8 → N9 → EXIT Recorre correctamente el bucle, modalidad ORDINARIA y resultado APROBADO.
- **P5** N1 → N2 → N3 → N4 → N5 → N3 → N6 → N7 → N8 → N9bis → EXIT Recorre correctamente el bucle, modalidad ORDINARIA y resultado SUSPENSO.
- **P6** N1 → N2 → N3 → N4 → N5 → N3 → N6 → N7 → N10 → N11 → EXIT Recorre correctamente el bucle, modalidad EXTRAORDINARIA y resultado APROBADO.
- **P7** N1 → N2 → N3 → N4 → N5 → N3 → N6 → N7 → N10 → N11bis → EXIT Recorre correctamente el bucle, modalidad EXTRAORDINARIA y resultado SUSPENSO.
- **P8** N1 → N2 → N3 → N4 → N5 → N3 → N6 → N7 → N12 → EXIT Recorrido normal del bucle, pero modalidad no válida (default del switch).
- **P9** N1 → N2 → N3 → N4 → N5 → N3 → N4 → N5 → N3 → N6 → N7 → N8 → N9 → EXIT Este camino introduce una característica nueva respecto a P4: **más de una iteración válida del bucle** antes de salir.

Es importante porque en el grafo el nodo N3 representa un ciclo, y uno de los caminos básicos debe reflejar ese comportamiento iterativo adicional.

Observación didáctica

En caminos con bucles, los caminos básicos no significan “todas las iteraciones posibles”, sino **variaciones independientes del control**. Por eso basta con reflejar:

- error en la primera iteración
- una iteración correcta y salida
- varias iteraciones correctas y salida

5. Casos de prueba

Caso	notas	modalidad	Resultado esperado	Camino
CP1	`null`	`"ORDINARIA"`	Excepción	P1
CP2	`{}`	`"ORDINARIA"`	Excepción	P2
CP3	`{7, -1, 8}`	`"ORDINARIA"`	Excepción	P3
CP4	`{6, 4}`	`"ORDINARIA"`	`"APROBADO"`	P4
CP5	`{3, 4}`	`"ORDINARIA"`	`"SUSPENSO"`	P5
CP6	`{4, 4}`	`"EXTRAORDINARIA"`	`"APROBADO"`	P6
CP7	`{2, 3}`	`"EXTRAORDINARIA"`	`"SUSPENSO"`	P7
CP8	`{8, 7}`	`"LIBRE"`	Excepción	P8
CP9	`{6, 7, 5}`	`"ORDINARIA"`	`"APROBADO"`	P9

Por qué CP4 y CP9 no son redundantes

A nivel funcional ambos dan "APROBADO" en ordinaria, pero estructural no recorren exactamente lo mismo:

- CP4: recorre el bucle con una iteración simple
- CP9: recorre el bucle con varias iteraciones válidas

Eso es útil para justificar el tratamiento del nodo del bucle en el grafo detallado.

6. JUnit 5 clásico

```java
import org.junit.jupiter.api.AfterAll;
import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeAll;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.*;

class EvaluadorAsignaturaTest {
    private static int contadorTests;

    @BeforeAll
    static void iniciarSuite() {
        contadorTests = 0;
        System.out.println("=== Inicio de tests de EvaluadorAsignatura ===");
    }

    @AfterAll
    static void cerrarSuite() {
        System.out.println("=== Fin de tests de EvaluadorAsignatura ===");
        System.out.println("Total de tests ejecutados: " + contadorTests);
    }

    @BeforeEach
    void prepararCaso() {
        contadorTests++;
    }

    @AfterEach
    void limpiarCaso() {
        // No hay recursos que liberar en este ejercicio
    }

    @Test
    void testCP1ArrayNuloLanzaExcepcion() {
        IllegalArgumentException ex = assertThrows(
                IllegalArgumentException.class,
                () -> EvaluadorAsignatura.evaluar(null, "ORDINARIA")
        );
        assertNotNull(ex);
        assertTrue(ex.getMessage().contains("nulo"));
    }
```

```java
    @Test
    void testCP2ArrayVacioLanzaExcepcion() {
        double[] notas = {};

        IllegalArgumentException ex = assertThrows(
                IllegalArgumentException.class,
                () -> EvaluadorAsignatura.evaluar(notas, "ORDINARIA")
        );
        assertNotNull(ex);
        assertTrue(ex.getMessage().contains("al menos una nota"));
    }

    @Test
    void testCP3NotaFueraDeRangoLanzaExcepcion() {
        double[] notas = {7, -1, 8};
        IllegalArgumentException ex = assertThrows(
                IllegalArgumentException.class,
                () -> EvaluadorAsignatura.evaluar(notas, "ORDINARIA")
        );
        assertEquals("Nota fuera de rango", ex.getMessage());
    }

    @Test
    void testCP4OrdinariaAprobado() {
        double[] notas = {6, 4};
        String resultado = EvaluadorAsignatura.evaluar(notas, "ORDINARIA");
        assertEquals("APROBADO", resultado);
        assertTrue(resultado.equals("APROBADO"));
        assertNotEquals("SUSPENSO", resultado);
    }

    @Test
    void testCP5OrdinariaSuspenso() {
        double[] notas = {3, 4};
        String resultado = EvaluadorAsignatura.evaluar(notas, "ORDINARIA");
        assertEquals("SUSPENSO", resultado);
        assertFalse(resultado.equals("APROBADO"));
    }

    @Test
    void testCP6ExtraordinariaAprobado() {
        double[] notas = {4, 4};
        String resultado = EvaluadorAsignatura.evaluar(notas, "EXTRAORDINARIA");
        assertEquals("APROBADO", resultado);
        assertNotNull(resultado);
    }

    @Test
    void testCP7ExtraordinariaSuspenso() {
        double[] notas = {2, 3};
        String resultado = EvaluadorAsignatura.evaluar(notas, "EXTRAORDINARIA");
        assertEquals("SUSPENSO", resultado);
        assertTrue(resultado.startsWith("SUS"));
    }

    @Test
    void testCP8ModalidadNoValidaLanzaExcepcion() {
        double[] notas = {8, 7};
        IllegalArgumentException ex = assertThrows(
                IllegalArgumentException.class,
                () -> EvaluadorAsignatura.evaluar(notas, "LIBRE")
        );
        assertNotNull(ex);
        assertTrue(ex.getMessage().contains("Modalidad"));
    }

    @Test
    void testCP9OrdinariaAprobadoConTresNotas() {
        double[] notas = {6, 7, 5};
        String resultado = EvaluadorAsignatura.evaluar(notas, "ORDINARIA");
        assertEquals("APROBADO", resultado);
        assertNotNull(resultado);
        assertFalse(resultado.isBlank());
    }
}
```

7. Cobertura esperada en IntelliJ IDEA

Si ejecutas esta clase con:

Run 'EvaluadorAsignaturaTest' with Coverage

deberías obtener una cobertura muy alta, y en la práctica completa para el método evaluar.

Qué queda cubierto

Validaciones iniciales

- notas == null
- notas.length == 0

Bucle

- entrada en el bucle
- salida del bucle
- una iteración
- varias iteraciones
- error dentro del bucle por nota inválida

switch

- rama ORDINARIA
- rama EXTRAORDINARIA
- rama default

Decisiones finales

- aprobado en ordinaria
- suspenso en ordinaria
- aprobado en extraordinaria
- suspenso en extraordinaria

Cobertura esperable

- **líneas**: 100%
- **métodos**: 100%
- **ramas**: 100% en el método evaluar

Observaciones metodológicas importantes

Con este enfoque ya aparecen varias cosas que antes faltaban:

- tests JUnit **clásicos**
- un método por caso de prueba
- variedad de aserciones
- uso real de anotaciones del ciclo de vida
- módulos más cercanos a lo que se suele programar en Java
- presencia de **switch** y **bucles**, no solo cascadas de if

Además, se puede ver que:

1. primero se analiza el código,
2. luego se construye el grafo,
3. después se calcula la complejidad,
4. luego se eligen caminos,
5. de ahí salen los casos de prueba,
6. y finalmente esos casos se implementan en JUnit.

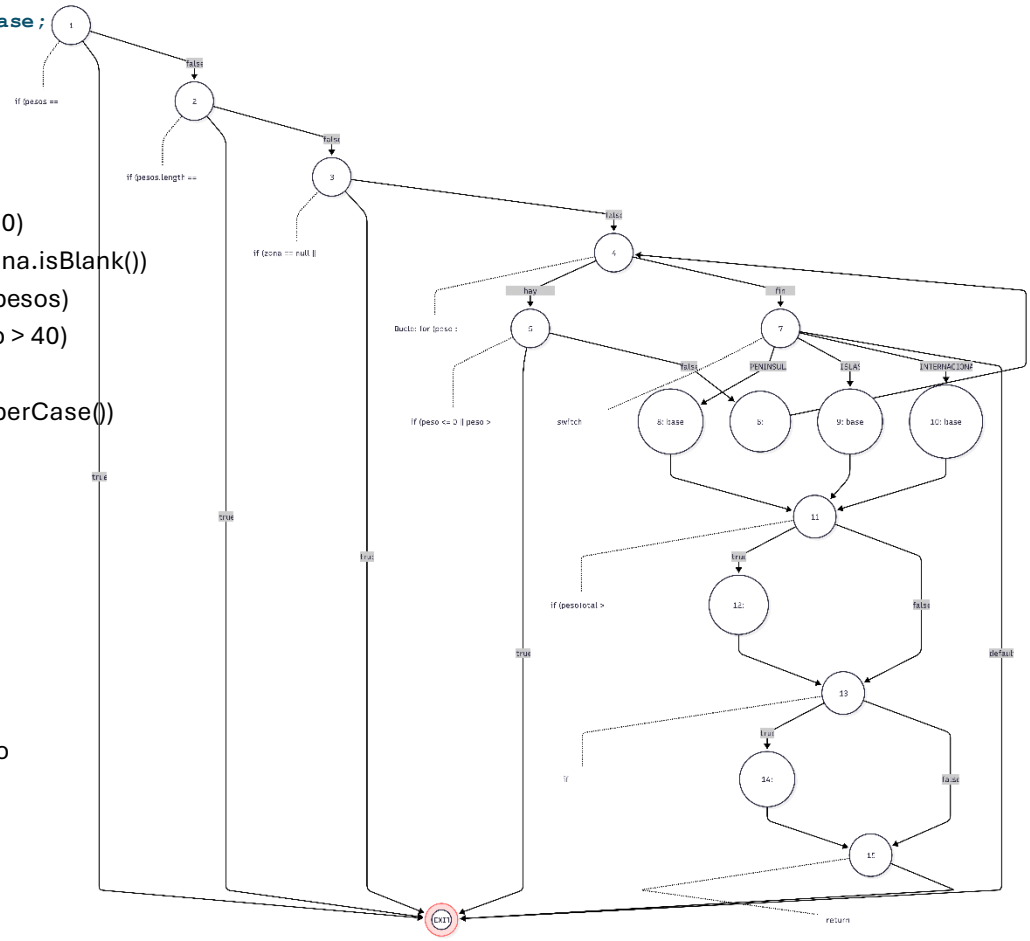
Ejercicio 3. Cálculo de coste de envío

1. Código Java

```java
public class CalculadoraEnvio {
    public static double calcularCoste(double[] pesos, String zona, boolean urgente) {
        if (pesos == null) {
            throw new IllegalArgumentException("La lista de pesos no puede ser nula");
        }
        if (pesos.length == 0) {
            throw new IllegalArgumentException("Debe existir al menos un paquete");
        }
        if (zona == null || zona.isBlank()) {
            throw new IllegalArgumentException("La zona no puede ser nula o vacía");
        }
        double pesoTotal = 0;
        for (double peso : pesos) {
            if (peso <= 0 || peso > 40) {
                throw new IllegalArgumentException("Peso fuera de rango");
            }
            pesoTotal += peso;
        }
        double costeBase;
        switch (zona.toUpperCase()) {
            case "PENINSULA":
                costeBase = 5;
                break;
            case "ISLAS":
                costeBase = 8;
                break;
            case "INTERNACIONAL":
                costeBase = 15;
                break;
            default:
                throw new IllegalArgumentException("Zona no válida");
        }
        if (pesoTotal > 20) {
            costeBase += 10;
        }
        if (urgente) {
            costeBase += 5;
        }
        return costeBase;
    }
}
```

2. Grafo de flujo

N1 → if (pesos == null)
N2 → if (pesos.length == 0)
N3 → if (zona == null || zona.isBlank())
N4 → for (double peso : pesos)
N5 → if (peso <= 0 || peso > 40)
N6 → pesoTotal += peso
N7 → switch (zona.toUpperCase())
N8 → costeBase = 5
N9 → costeBase = 8
N10 → costeBase = 15
N11 → if (pesoTotal > 20)
N12 → costeBase += 10
N13 → if (urgente)
N14 → costeBase += 5
N15 → return costeBase
EXIT → salida del método



3. Complejidad ciclomática

Usamos la fórmula de McCabe: $V(G) = E - N + 2P$

Como el grafo tiene un único componente conexo: $P = 1$

Sustituyendo: $V(G) = 25 - 16 + 2 = 11$

Resultado $\boxed{V(G) = 11}$

Conteo exacto del grafo que has dibujado

Tu grafo:

- separa explícitamente las ramas del switch
- incluye un default que va a EXIT
- separa nodos de acumulación y recargos
- explicita el ciclo del bucle con detalle
- Por eso la complejidad del **grafo detallado** es mayor: $\boxed{11}$

4. Caminos básicos

A partir de $V(G) = 11$, un conjunto válido de **11 caminos independientes** puede ser este.

- **P1** $N1 \rightarrow EXIT$ pesos == null
- **P2** $N1 \rightarrow N2 \rightarrow EXIT$ pesos no es nulo, pero pesos.length == 0
- **P3** $N1 \rightarrow N2 \rightarrow N3 \rightarrow EXIT$ pesos válidos, pero zona == null || isBlank
- **P4** $N1 \rightarrow N2 \rightarrow N3 \rightarrow N4 \rightarrow N5 \rightarrow EXIT$ entra en el bucle y aparece un peso fuera de rango
- **P5** $N1 \rightarrow N2 \rightarrow N3 \rightarrow N4 \rightarrow N5 \rightarrow N6 \rightarrow N4 \rightarrow N7 \rightarrow N8 \rightarrow N11 \rightarrow N13 \rightarrow N15 \rightarrow EXIT$ camino válido por PENINSULA, sin recargo por peso y sin urgencia
- **P6** $N1 \rightarrow N2 \rightarrow N3 \rightarrow N4 \rightarrow N5 \rightarrow N6 \rightarrow N4 \rightarrow N7 \rightarrow N8 \rightarrow N11 \rightarrow N12 \rightarrow N13 \rightarrow N15 \rightarrow EXIT$
- camino válido por PENINSULA, con recargo por peso, sin urgencia
- **P7** $N1 \rightarrow N2 \rightarrow N3 \rightarrow N4 \rightarrow N5 \rightarrow N6 \rightarrow N4 \rightarrow N7 \rightarrow N8 \rightarrow N11 \rightarrow N13 \rightarrow N14 \rightarrow N15 \rightarrow EXIT$
- camino válido por PENINSULA, sin recargo por peso, con urgencia
- **P8** $N1 \rightarrow N2 \rightarrow N3 \rightarrow N4 \rightarrow N5 \rightarrow N6 \rightarrow N4 \rightarrow N7 \rightarrow N9 \rightarrow N11 \rightarrow N13 \rightarrow N15 \rightarrow EXIT$
- camino válido por ISLAS, sin recargo por peso y sin urgencia
- **P9** $N1 \rightarrow N2 \rightarrow N3 \rightarrow N4 \rightarrow N5 \rightarrow N6 \rightarrow N4 \rightarrow N7 \rightarrow N10 \rightarrow N11 \rightarrow N13 \rightarrow N15 \rightarrow EXIT$
- camino válido por INTERNACIONAL, sin recargo por peso y sin urgencia
- **P10** $N1 \rightarrow N2 \rightarrow N3 \rightarrow N4 \rightarrow N5 \rightarrow N6 \rightarrow N4 \rightarrow N7 \rightarrow EXIT$
- default del switch, zona no válida
- **P11** $N1 \rightarrow N2 \rightarrow N3 \rightarrow N4 \rightarrow N5 \rightarrow N6 \rightarrow N4 \rightarrow N5 \rightarrow N6 \rightarrow N4 \rightarrow N7 \rightarrow N10 \rightarrow N11 \rightarrow N12 \rightarrow N13 \rightarrow N14 \rightarrow N15 \rightarrow EXIT$ camino con **varias iteraciones válidas del bucle**, rama INTERNACIONAL, con recargo por peso y con urgencia

Observación importante sobre estos caminos

Aquí conviene ser muy preciso:

- **P5, P6 y P7** distinguen tres combinaciones distintas dentro de la rama PENINSULA
- **P8** introduce la rama ISLAS
- **P9** introduce la rama INTERNACIONAL
- **P10** cubre el default
- **P11** introduce una variación independiente del bucle: más de una iteración antes de salir

Eso justifica que el conjunto llegue a 11.

5. Casos de prueba derivados del grafo

Caso	pesos	Zona	urgente	Resultado esperado	Camino
CP1	null	"PENINSULA"	false	Excepción	P1
CP2	{}	"PENINSULA"	false	Excepción	P2
CP3	{2,3}	""	false	Excepción	P3
CP4	{2,-1,5}	"ISLAS"	false	Excepción	P4
CP5	{2,3,4}	"PENINSULA"	false	5.0	P5
CP6	{10,12}	"PENINSULA"	false	15.0	P6
CP7	{2,3,4}	"PENINSULA"	true	10.0	P7
CP8	{3,4}	"ISLAS"	false	8.0	P8
CP9	{5,5}	"INTERNACIONAL"	false	15.0	P9
CP10	{2,3}	"EUROPA"	false	Excepción	P10
CP11	{15,10}	"INTERNACIONAL"	True	30.0	P11

Comprobación de algunos valores

- **CP5**: Península base 5, peso total 9, no urgente → **5**
- **CP6**: Península base 5, peso total 22, +10, no urgente → **15**
- **CP7**: Península base 5, peso total 9, urgente +5 → **10**
- **CP8**: Islas base 8, peso total 7, no urgente → **8**
- **CP9**: Internacional base 15, peso total 10, no urgente → **15**
- **CP11**: Internacional base 15, peso total 25, +10, urgente +5 → **30**

7. JUnit 5 Parametrizado

Idea general de la parametrización

En este ejercicio, a partir del grafo detallado, obteníamos: $V(G) = 11$

Por tanto, necesitamos 11 casos de prueba mínimos.

Sin embargo, no es necesario escribir 11 métodos @Test distintos, porque muchos casos siguen exactamente el mismo patrón:

Grupo A. Casos válidos

En todos ellos:

se llama al método calcularCoste(…)

se obtiene un double

se compara con el valor esperado mediante assertEquals

Grupo B. Casos excepcionales

En todos ellos:

se llama al método calcularCoste(…)

se espera una IllegalArgumentException

se comprueba el mensaje con assertThrows

Eso permite parametrizar los datos de cada caso y reutilizar la misma lógica de test.

La idea clave es esta:

No se parametriza la lógica del test; se parametrizan **los datos del caso de prueba.**

Por qué usamos @MethodSource

Aquí intervienen arrays double[], y eso hace que @CsvSource no sea la mejor opción.

@MethodSource es más adecuado porque permite construir cada caso libremente con:

Arguments.of(…)

y pasar sin problema:

- arrays
- cadenas
- booleanos
- números reales
- mensajes esperados

```java
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.MethodSource;
import org.junit.jupiter.params.provider.Arguments;
import java.util.stream.Stream;
import static org.junit.jupiter.api.Assertions.*;

class CalculadoraEnvioTest {
    // ------------------------------------------------
    // CASOS VÁLIDOS
    // ------------------------------------------------
    static Stream<Arguments> casosValidos() {
        return Stream.of(
                Arguments.of("CP5 - Península, peso normal, no urgente",
                        new double[]{2, 3, 4}, "PENINSULA", false, 5.0),

                Arguments.of("CP6 - Península, peso alto, no urgente",
                        new double[]{10, 12}, "PENINSULA", false, 15.0),

                Arguments.of("CP7 - Península, peso normal, urgente",
                        new double[]{2, 3, 4}, "PENINSULA", true, 10.0),

                Arguments.of("CP8 - Islas, peso normal, no urgente",
                        new double[]{3, 4}, "ISLAS", false, 8.0),

                Arguments.of("CP9 - Internacional, peso normal, no urgente",
                        new double[]{5, 5}, "INTERNACIONAL", false, 15.0),

                Arguments.of("CP11 - Internacional, peso alto, urgente",
                        new double[]{15, 10}, "INTERNACIONAL", true, 30.0)
        );
    }
    @ParameterizedTest(name = "{0}")
    @MethodSource("casosValidos")
    void testCasosValidos(String nombreCaso,
                          double[] pesos,
                          String zona,
                          boolean urgente,
                          double esperado) {

        double resultado = CalculadoraEnvio.calcularCoste(pesos, zona, urgente);

        assertEquals(esperado, resultado);
    }

    // ------------------------------------------------
    // CASOS EXCEPCIONALES
    // ------------------------------------------------
    static Stream<Arguments> casosExcepcionales() {
        return Stream.of(
                Arguments.of("CP1 - Pesos nulos",
                        null, "PENINSULA", false, "nula"),
                Arguments.of("CP2 - Array vacío",
                        new double[]{}, "PENINSULA", false, "al menos un paquete"),
                Arguments.of("CP3 - Zona vacía",
                        new double[]{2, 3}, "", false, "zona"),
                Arguments.of("CP4 - Peso fuera de rango",
                        new double[]{2, -1, 5}, "ISLAS", false, "Peso fuera de rango"),
                Arguments.of("CP10 - Zona inválida en switch",
                        new double[]{2, 3}, "EUROPA", false, "Zona no válida")
        );
    }
    @ParameterizedTest(name = "{0}")
    @MethodSource("casosExcepcionales")
    void testCasosExcepcionales(String nombreCaso,
                                double[] pesos,
                                String zona,
                                boolean urgente,
                                String textoEsperadoEnMensaje) {
        IllegalArgumentException ex = assertThrows(
                IllegalArgumentException.class,
                () -> CalculadoraEnvio.calcularCoste(pesos, zona, urgente)
        );
        assertTrue(ex.getMessage().contains(textoEsperadoEnMensaje));
    }
}
```

8. Explicación detallada de cómo se parametriza

8.1. Fuente de datos para casos válidos

Esta parte:

```
static Stream<Arguments> casosValidos() {
    return Stream.of(
        Arguments.of("CP5 ...", new double[]{2, 3, 4}, "PENINSULA", false, 5.0),
        ...
    );
}
```

define una **colección de casos de prueba**.

Cada Arguments.of(...) representa un caso completo con sus datos:

1. nombre del caso
2. array de pesos
3. zona
4. urgencia
5. valor esperado

JUnit tomará cada fila y ejecutará el método testCasosValidos(...) una vez con esos datos.

8.2. Método de test parametrizado de casos válidos

```
@ParameterizedTest(name = "{0}")
@MethodSource("casosValidos")
void testCasosValidos(String nombreCaso,
                      double[] pesos,
                      String zona,
                      boolean urgente,
                      double esperado) {
    double resultado = CalculadoraEnvio.calcularCoste(pesos, zona, urgente);
    assertEquals(esperado, resultado);
}
```

Qué ocurre aquí

- @ParameterizedTest indica que no es un test simple, sino un test que se ejecutará varias veces.
- @MethodSource("casosValidos") dice que los datos vendrán del método casosValidos().
- name = "{0}" hace que JUnit muestre como nombre visible el primer parámetro, es decir, el texto del caso.

De esta forma, **un solo método** reemplaza a 6 métodos casi idénticos.

8.3. Fuente de datos para casos excepcionales

Lo mismo ocurre con los casos que deben lanzar excepción:

```
static Stream<Arguments> casosExcepcionales() {
    return Stream.of(
        Arguments.of("CP1 ...", null, "PENINSULA", false, "nula"),
        ...
    );
}
```

Aquí cada caso contiene:

1. nombre del caso
2. pesos
3. zona
4. urgencia
5. fragmento del mensaje esperado

8.4. Método de test parametrizado de excepciones

```
@ParameterizedTest(name = "{0}")
@MethodSource("casosExcepcionales")
void testCasosExcepcionales(String nombreCaso,
                            double[] pesos,
                            String zona,
                            boolean urgente,
                            String textoEsperadoEnMensaje) {

    IllegalArgumentException ex = assertThrows(
            IllegalArgumentException.class,
            () -> CalculadoraEnvio.calcularCoste(pesos, zona, urgente)
    );

    assertTrue(ex.getMessage().contains(textoEsperadoEnMensaje));
}
```

Aquí:

- assertThrows(...) comprueba que efectivamente se lance la excepción
- luego se valida el contenido del mensaje con assertTrue(...)

De nuevo, **un solo método** reemplaza a 5 tests repetitivos.

9. Qué se simplifica realmente

Sin parametrización

Habría que escribir:

- 6 métodos @Test para casos válidos
- 5 métodos @Test para excepciones

Total:

- **11 métodos de test**

Y muchos de ellos tendrían exactamente la misma estructura.

Con parametrización

Se reduce a:

- 2 métodos parametrizados
- 2 fuentes de datos

La lógica del test se escribe una sola vez para cada patrón.

Ventajas

- menos duplicación
- más mantenible
- más fácil añadir nuevos casos
- más limpio visualmente
- sigue existiendo trazabilidad con los caminos básicos

10. Correspondencia entre caminos y test parametrizado

Aunque el código esté simplificado, **no se pierde la relación con el grafo**.

Casos válidos

Camino	Caso	Fuente
P5	CP5	casosValidos()
P6	CP6	casosValidos()
P7	CP7	casosValidos()
P8	CP8	casosValidos()
P9	CP9	casosValidos()
P11	CP11	casosValidos()

Casos excepcionales

Camino	Caso	Fuente
P1	CP1	casosExcepcionales()
P2	CP2	casosExcepcionales()
P3	CP3	casosExcepcionales()
P4	CP4	casosExcepcionales()
P10	CP10	casosExcepcionales()

Es decir, los **11 casos mínimos siguen ahí**.

Lo único que cambia es la forma de implementarlos en JUnit.